# Reporte de Funcionamiento de IoT Server Frontend

Este documento presenta los resultados de las pruebas de funcionalidad, seguridad y responsividad del frontend del sistema IoT Server. El análisis incluye:

- Pruebas de funcionalidad: Verificación de todas las rutas, componentes CRUD y endpoints
- Pruebas de seguridad OWASP: Análisis de vulnerabilidades comunes del TOP 10 de OWASP
- Pruebas de responsividad: Verificación del diseño adaptativo en diferentes dispositivos
- Diagramas de flujo: Documentación visual de los procesos principales

## Pruebas de Funcionalidad

- Autenticación (POST /api/v1/auth/login)

Endpoint testeado: `POST /api/v1/auth/login`

*Casos de prueba:*

| Entrada | Resultado esperado | Estado |
|---|---|---|
| Email + contraseña válidos (Admin Master) | JWT token + `type: "administrator"` + `is_master: true` | Funcional (por código) |
| Email + contraseña válidos (Admin Normal) | JWT token + `type: "administrator"` + `is_master: false` | Funcional (por código) |
| Email + contraseña válidos (Gerente) | JWT token + `type: "manager"` | Funcional (por código) |
| Email + contraseña válidos (Usuario) | \| JWT token + `type: "user"` | Funcional (por código) |
| Email incorrecto | 401 Unauthorized` — "Credenciales inválidas" | Manejado |
| Contraseña incorrecta | `401 Unauthorized` — "Credenciales inválidas" | Manejado |
| Email vacío | Validación Yup bloquea submit → "Email requerido" | Validado |
| Contraseña vacía | Validación Yup bloquea submit → "Contraseña requerida" | Validado |
| Email con formato inválido | Validación Yup → "Email inválido" | Validado |
| Token expirado (60 min) | Redirección automática a `/login/admin-master` | Implementado |
| Token manipulado | Backend responde `401`, frontend muestra error | Manejado |

- Almacenamiento de sesión:
  - Token JWT guardado en `sessionStorage` (key: `"iot-auth"`)
  - Decodificación en frontend para extraer `sub` (ID), `type`, `is_master`, `exp`
  - Vulnerabilidad potencial: `sessionStorage` es accesible por JavaScript → vulnerable a XSS si se inyecta código malicioso (ver sección OWASP)

**Gestión de Administradores (Solo Admin Master)**

- Endpoints testeados:

| Operación | Método | Endpoint | Permisos |

| Listar | GET | `/api/v1/administrators/?page=1&per_page=50` | Admin Master |

| Crear | POST | `/api/v1/administrators/` | Admin Master |

| Editar | PATCH | `/api/v1/administrators/{id}` | Admin Master |

| Eliminar | DELETE | `/api/v1/administrators/{id}` | Admin Master |

| Ver detalle | GET | (usa datos de lista) | Admin Master |

- Casos de prueba:

| # | Operación | Entrada | Resultado esperado | Estado |

| 1 | Listar | Sesión Admin Master | Lista paginada con administradores | ✅ Implementado |

| 2 | Crear | Formulario válido con todos los campos | `201 Created` → Nuevo admin en lista | ✅ Implementado |

| 3 | Crear | Email duplicado | `400 Bad Request` → "Ya existe un registro con ese email" | ✅ Manejado |

| 4 | Crear | CURP duplicado | `400 Bad Request` → "Ya existe un registro con ese curp" | ✅ Manejado |

| 5 | Crear | Formato de email inválido | Validación Yup bloquea submit | ✅ Validado |

| 6 | Editar | Cambiar `first_name` | `200 OK` → Nombre actualizado en tabla | ✅ Implementado |

| 7 | Editar | Cambiar `is_active` a `false` | `200 OK` → Estado reflejado en tabla | ✅ Implementado |

| 8 | Editar | Intentar cambiar email a uno existente | `400 Bad Request` → Error mostrado | ✅ Manejado |

| 9 | Eliminar | Admin con ID diferente al actual | `204 No Content` → Fila removida de tabla | ✅ Implementado |

| 10 | Eliminar | Intentar eliminar a sí mismo | Frontend previene: botón DELETE deshabilitado si `row.id === selfId` | ✅ Protegido |

| 11 | Eliminar | Sin razón de eliminación | Diálogo de confirmación bloquea → "Razón requerida (≥3 caracteres)" | ✅ Validado |

| 12 | Ver detalle | Click en ícono 👁 | Diálogo muestra todos los campos (sensibles enmascarados) | ✅ Implementado |

| 13 | Acceso deshabilita | Sesión de Admin Normal intenta acceder | `ProtectedRoute` redirige a `/` (Dashboard) | ✅ Protegido |

| 14 | Acceso sin autenticación | Sin token JWT → Redirige a `/login/admin-master` | ✅ Protegido |

**Gestión de Gerentes (Admin)**

- Endpoints testeados:

| Operación | Método | Endpoint | Permisos |

| Listar | GET | `/api/v1/managers/?page=1&per_page=50` | Todos autenticados |

| Crear | POST | `/api/v1/managers/` | Admin |

| Editar | PATCH | `/api/v1/managers/{id}` | Admin |

| Eliminar | DELETE | `/api/v1/managers/{id}` | Admin |

- Casos de prueba:

| # | Operación | Entrada | Resultado esperado | Estado |

| 1 | Listar | Cualquier sesión autenticada | Lista paginada | ✅ Funcional |

| 2 | Crear | Sesión Admin + formulario válido | `201 Created` | ✅ Funcional |

| 3 | Crear | Sesión Manager intenta crear | Frontend bloquea botón "Crear" (`canModify = false`) | ✅ UI protegida |

| 4 | Crear | Email duplicado | `400 Bad Request` | ✅ Manejado |

| 5 | Editar | Admin edita gerente | `200 OK` | ✅ Funcional |

| 6 | Editar | Manager intenta editar | UI bloquea (sin botón editar) | ✅ UI protegida |

| 7 | Eliminar | Admin elimina gerente + razón válida | `204 No Content` | ✅ Funcional |

| 8 | Eliminar | Manager intenta eliminar | UI bloquea (sin botón eliminar) | ✅ UI protegida |

⚠️ **Problema de seguridad backend**: El backend NO valida permisos en `/api/v1/managers/` — cualquier usuario autenticado (incluso "user") puede crear/editar/eliminar gerentes si conoce el endpoint. El frontend protege la UI, pero un atacante puede hacer peticiones directas con cURL/Postman.

Recomendación: Agregar decoradores de autorización OSO en `app/domain/manager/controller.py

**Gestión de Usuarios (Admin y Manager)**

- Endpoints testeados:

| Operación | Método | Endpoint | Permisos |

| Listar | GET | `/api/v1/users/?page=1&per_page=50` | Admin, Manager |

| Crear | POST | `/api/v1/users/` | Admin |

| Editar | PATCH | `/api/v1/users/{id}` | Admin |

| Eliminar | DELETE | `/api/v1/users/{id}` | Admin |

- Casos de prueba:

| # | Operación | Entrada | Resultado esperado | Estado |

| 1 | Listar | Sesión Admin o Manager | Lista paginada usuarios | ✅ Funcional |

| 2 | Listar | Sesión User intenta listar | `ProtectedRoute` bloquea acceso a `/usuarios` | ✅ Protegido |

| 3 | Crear | Admin + formulario válido | `201 Created` | ✅ Funcional |

| 4 | Crear | Manager intenta crear | UI bloquea (botón "Crear" oculto si `accountType !== "administrator"`) | ✅ UI protegida |

| 5 | Editar | Admin edita usuario | `200 OK` | ✅ Funcional |

| 6 | Editar | Manager intenta editar | UI bloquea | ✅ UI protegida |

| 7 | Eliminar | Admin elimina usuario | `204 No Content` | ✅ Funcional |

| 8 | Eliminar | Manager intenta eliminar | UI bloquea (`canDelete = false` para manager) | ✅ UI protegida |

| 9 | Editar campo `is_active` | Admin cambia estado a `false` | Toggle en tabla → `PATCH` → Estado actualizado | ✅ Funcional |

| 10 | Ver detalle sensible | Click en 👁 → Ver CURP/RFC | Campos enmascarados: `••••••••` | ✅ Protegido |

| 11 | Editar campo sensible | Formulario editar → Campo `curp` con toggle 👁 | Por defecto enmascarado, toggle para revelar | ✅ Implementado |

**Gestión de Dispositivos (Todos autenticados)**

- Endpoints testeados:

| Operación | Método | Endpoint | Permisos |

| Listar | GET | `/api/v1/devices/?page=1&per_page=50` | Todos autenticados |

| Crear | POST | `/api/v1/devices/` | Admin (UI protege) |

| Editar | PATCH | `/api/v1/devices/{id}` | Admin (UI protege) |

| Eliminar | DELETE | `/api/v1/devices/{id}` | Admin (UI protege) |

- Casos de prueba:

| # | Operación | Entrada | Resultado esperado | Estado |

| 1 | Listar | Cualquier sesión | Lista de dispositivos | ✅ Funcional |

| 2 | Crear | Admin + formulario válido | `201 Created` | ✅ Funcional |

| 3 | Crear | IP con formato inválido (e.g., "999.999.999.999") | Backend `422` → "Formato IP inválido" | ✅ Validado (backend) |

| 4 | Crear | MAC address inválida (e.g., "ZZ:ZZ:ZZ:ZZ:ZZ:ZZ") | Backend `422` → "Formato MAC inválido" | ✅ Validado (backend) |

| 5 | Crear | Manager o User intenta crear | UI bloquea botón "Crear" | ✅ UI protegida |

| 6 | Editar | Admin edita `name` | `200 OK` | ✅ Funcional |

| 7 | Editar | Manager intenta editar | UI sin botón editar | ✅ UI protegida |

| 8 | Eliminar | Admin elimina dispositivo | `204 No Content` | ✅ Funcional |

| 9 | Editar `is_active` | Admin toggle estado en tabla | `PATCH` → Actualización inmediata | ✅ Funcional |

| 10 | Ver detalle | Click 👁 → Ver IP, MAC, ubicación | Detalles completos mostrados | ✅ Funcional |

⚠️ **Problema de seguridad backend:** Similar a gerentes, el backend NO valida permisos en `/api/v1/devices/` — cualquier usuario autenticado puede crear/editar/eliminar dispositivos vía API directa.

**Gestión de Servicios (Admin y Manager)**

- Endpoints testeados:

| Operación | Método | Endpoint | Permisos |

| Listar | GET | `/api/v1/services/?page=1&per_page=50` | Admin, Manager |

| Crear | POST | `/api/v1/services/` | Admin |

| Editar | PATCH | `/api/v1/services/{id}` | Admin |

| Eliminar | DELETE | `/api/v1/services/{id}` | Admin |

- Casos de prueba:

| # | Operación | Entrada | Resultado esperado | Estado |

| 1 | Listar | Sesión Admin o Manager | Lista de servicios | ✅ Funcional |

| 2 | Crear | Admin + nombre + descripción | `201 Created` | ✅ Funcional |

| 3 | Crear | Manager intenta crear | UI bloquea botón | ✅ UI protegida |

| 4 | Editar | Admin edita servicio | `200 OK` | ✅ Funcional |

| 5 | Eliminar | Admin elimina servicio | `204 No Content` | ✅ Funcional |

| 6 | Ver detalle | Click 👁 → Ver detalles | Información completa mostrada | ✅ Funcional |

**Gestión de Aplicaciones (Admin y Manager)**

- Endpoints testeados:

| Operación | Método | Endpoint | Permisos |

| Listar | GET | `/api/v1/applications/?page=1&per_page=50` | Admin, Manager |

| Crear | POST | `/api/v1/applications/` | Admin |

| Editar | PATCH | `/api/v1/applications/{id}` | Admin |

| Eliminar | DELETE | `/api/v1/applications/{id}` | Admin |

- Casos de prueba:

| # | Operación | Entrada | Resultado esperado | Estado |

| 1 | Listar | Sesión Admin o Manager | Lista de aplicaciones IoT | ✅ Funcional |

| 2 | Crear | Admin + campos obligatorios + opcionales vacíos | `201 Created` (campos opcionales usan defaults backend) | ✅ Funcional |

| 3 | Crear | `api_key` incluida en formulario | Campo `api_key` NO se envía (auto-generado por backend) | ✅ Protegido |

| 4 | Editar | Admin edita `name` | `200 OK` | ✅ Funcional |

| 5 | Eliminar | Admin elimina app | `204 No Content` | ✅ Funcional |

| 6 | Ver `api_key` | Ver detalle de aplicación | `api_key` enmascarada: `••••••••` | ✅ Protegido |

| 7 | Editar `api_key` | Formulario editar → Campo `api_key` con toggle | Por defecto enmascarado, revelable con 👁 | ✅ Implementado |

**Gestión de Tickets (Todos autenticados)**

- Endpoints testeados:**

| Tipo | Operación | Método | Endpoint | Estado |

| Service Ticket | Listar | GET | `/api/v1/service-tickets/` | ✅ Funcional |

| Service Ticket | Crear | POST | `/api/v1/service-tickets/` | ✅ Funcional |

| Service Ticket | Editar | PATCH | `/api/v1/service-tickets/{id}` | ✅ Funcional |

| Service Ticket | Eliminar | DELETE | `/api/v1/service-tickets/{id}` | ✅ Funcional |

| Ecosystem Ticket | Listar | GET | `/api/v1/ecosystem-tickets/` | ✅ Funcional |

| Ecosystem Ticket | Crear | POST | `/api/v1/ecosystem-tickets/` | ✅ Funcional |

| Ecosystem Ticket | Editar | PATCH | `/api/v1/ecosystem-tickets/{id}` | ✅ Funcional |

| Ecosystem Ticket | Eliminar | DELETE | `/api/v1/ecosystem-tickets/{id}` | ✅ Funcional

- Casos de prueba:

| # | Operación | Entrada | Resultado esperado | Estado |

| 1 | Crear Service Ticket | Formulario con select de `priority` y `status_id` | `201 Created` | ✅ Funcional |

| 2 | Priority select | Opciones: Baja / Media / Alta / Crítica | Valores: `low`, `medium`, `high`, `critical` | ✅ Enum correcto |

| 3 | Status select | Opciones: Abierto (1) / En progreso (2) / Resuelto (3) / Cerrado (4) | IDs numéricos enviados correctamente | ✅ Funcional |

| 4 | Service select | Select cargado desde `/api/v1/services/` | Nombres de servicios mostrados (UUID enviado) | ✅ Funcional |

| 5 | User Role ID | Campo texto libre (no hay select) | UUID requerido (validación Yup) | ⚠️ UX pobre — usuario debe conocer UUID |

| 6 | Manager Service ID | Campo texto libre | UUID requerido | ⚠️ UX pobre |

| 7 | Ver detalle | Click 👁 → Ver ticket completo | Todos los campos mostrados | ✅ Funcional |

| 8 | Editar ticket | Cambiar `priority` de Media a Alta | `200 OK` → Prioridad actualizada | ✅ Funcional |

| 9 | Eliminar ticket | Admin elimina + razón | `204 No Content` | ✅ Funcional |

⚠️ **Funcionalidad faltante:** No hay endpoints para:

- `GET /api/v1/user-roles/` → Imposible cargar select de User Roles
- `GET /api/v1/manager-services/` → Imposible cargar select de Manager Services

*Solución futura: Agregar estos endpoints al backend o usar inputs con autocompletado que busquen por nombre.*

**Dashboard**

- Endpoint de estadísticas: GET individual para cada entidad

| Estadística | Endpoint | Permisos | Estado |

| Total dispositivos | `GET /api/v1/devices/?page=1&per_page=1` | Todos | ✅ Funcional |

| Total usuarios | `GET /api/v1/users/?page=1&per_page=1` | Admin, Manager | ✅ Funcional |

| Total administradores | `GET /api/v1/administrators/?page=1&per_page=1` | Admin Master | ✅ Funcional |

| Total gerentes | `GET /api/v1/managers/?page=1&per_page=1` | Todos | ✅ Funcional |

- Casos de prueba:

| # | Funcionalidad | Entrada | Resultado esperado | Estado |

| 1 | Ver contador dispositivos | Cualquier sesión | Número total visible | ✅ Funcional |

| 2 | Ver contador usuarios | Sesión Admin o Manager | Número total visible | ✅ Funcional |

| 3 | Ver contador usuarios | Sesión User | Card oculta (no se muestra) | ✅ Protegido |

| 4 | Ver contador admins | Sesión Admin Master | Número total visible | ✅ Funcional |

| 5 | Ver contador admins | Sesión Admin Normal | Card oculta | ✅ Protegido |

| 6 | Feed de actividad | Acción CRUD realizada | Nueva entrada aparece en feed inmediatamente | ✅ Funcional |

| 7 | Feed persistencia | Cerrar sesión → Reloguear | Actividades persisten (localStorage) | ✅ Funcional |

| 8 | Feed icono color | Crear entidad | Ícono verde 🟢 | ✅ Funcional |

| 9 | Feed icono color | Editar entidad | Ícono azul 🔵 | ✅ Funcional |

| 10 | Feed icono color | Eliminar entidad | Ícono rojo 🔴 | ✅ Funcional |

| 11 | Panel "Crear nuevo" | Click "Nuevo Usuario" | Redirige a `/usuarios` con botón crear habilitado | ✅ Funcional |

| 12 | Panel "Gestionar" | Click "Dispositivos" | Redirige a `/dispositivos` | ✅ Funcional |

| 13 | Permisos panel crear | Sesión User → "Nuevo Servicio" | Opción oculta (solo Admin) | ✅ Protegido |

| 14 | Permisos panel gestionar | Sesión Admin Normal → "Administradores" | Opción oculta (solo Admin Master) | ✅ Protegido |

**Cambio de contraseña**

Endpoint: `PATCH /api/v1/auth/change-password`

- Casos de prueba:

| # | Entrada | Resultado esperado | Estado |

| 1 | Contraseña actual correcta + nueva válida + confirmar coincide | `200 OK` → "Contraseña actualizada" | ✅ Funcional |

| 2 | Contraseña actual incorrecta | `401 Unauthorized` → "Contraseña actual incorrecta" | ✅ Manejado |

| 3 | Nueva contraseña < 8 caracteres | Validación Yup bloquea submit | ✅ Validado |

| 4 | Nueva ≠ Confirmar | Validación Yup → "Las contraseñas no coinciden" | ✅ Validado |

| 5 | Campos vacíos | Validación bloquea submit → "Campo requerido" | ✅ Validado |

| 6 | Intentar cambiar sin token JWT | Backend `401` → Redirige a login | ✅ Protegido |

**Logout**

*Implementación: Client-side únicamente (sin endpoint backend `/logout`)*

- Casos de prueba:

| # | Acción | Resultado esperado | Estado |

| 1 | Click en ícono 🔒 (AppBar) | Limpia `sessionStorage` → Redirige a `/login/admin-master` | ✅ Funcional |

| 2 | Verificar limpieza | `sessionStorage.getItem("iot-auth")` después de logout | `null` | ✅ Limpiado |

| 3 | Intentar acceder a ruta protegida después de logout | `ProtectedRoute` redirige a login | ✅ Protegido |

⚠️ **Limitación:** No hay invalidación del token en el servidor. Un token JWT robado seguirá siendo válido hasta su expiración (60 minutos) incluso después del logout. El backend debería implementar una blacklist de tokens o usar refresh tokens.

---

**Pruebas de Seguridad OWASP Top 10**

- Broken Access Control (Control de Acceso Roto)

Vulnerabilidades encontradas:

  - Backend sin autorización en endpoints críticos** ⚠️ **CRÍTICO**

| Endpoint | Método | Problema | Solución |

| `/api/v1/managers/` | POST, PATCH, DELETE | Cualquier usuario autenticado puede modificar gerentes | Agregar `@require_write("Manager")` |

| `/api/v1/devices/` | POST, PATCH, DELETE | Cualquier usuario puede crear/editar/eliminar dispositivos | Agregar decoradores de autorización |

| `/api/v1/services/` | POST, PATCH, DELETE | Sin validación de permisos | Agregar guards OSO |

  - Prueba de concepto:

```
# 1. Login como usuario normal

curl -X POST http://localhost:8000/api/v1/auth/login \

  -H "Content-Type: application/json" \

  -d '{"email":"user@test.com", "password":"pass123"}'


# 2. Crear gerente directamente (debería fallar pero no lo hace)

curl -X POST http://localhost:8000/api/v1/managers/ \

  -H "Authorization: Bearer <TOKEN_USER>" \

  -H "Content-Type: application/json" \

  -d '{"first_name":"Hacker","last_name":"Manager", ...}'

# ❌ RESULTADO: 201 Created (VULNERABLE)
```

**Estado:** ❌ **VULNERABLE** — El frontend protege la UI, pero el backend no valida permisos.

- Auto-eliminación solo protegida en frontend** ⚠️ **MEDIO**

**Problema:** Un administrador puede eliminar su propia cuenta haciendo una petición `DELETE` directa al endpoint `/api/v1/administrators/{self_id}`, aunque el frontend lo prevenga.

**Código vulnerable (backend):**

```
# app/domain/administrator/controller.py

@router.delete("/{administrator_id}")

async def delete_administrator(administrator_id: UUID, ...):

  # ❌ No valida si administrator_id == current_user_id

  await service.delete(administrator_id, session)

  return Response(status_code=204)
```

**Solución backend:**

```
from fastapi import HTTPException

if str(administrator_id) == str(current_account.personal_data_id):

  raise HTTPException(403, "No puedes eliminar tu propia cuenta")
```

**Estado: ⚠️ PARCIALMENTE PROTEGIDO (solo frontend)**

- IDOR (Insecure Direct Object Reference)** ✅ **NO VULNERABLE**

**Prueba:** Intentar acceder a entidades de otros usuarios manipulando IDs en la URL.

```
# Editar administrador con ID diferente

curl -X PATCH http://localhost:8000/api/v1/administrators/OTRO_UUID \

 -H "Authorization: Bearer <TOKEN_ADMIN_NORMAL>" \

 -H "Content-Type: application/json" \
```

-d '{"first_name":"Hacked"}'

**Resultado:** ✅ El backend valida permisos correctamente en endpoints de Administradores (requiere Admin Master).

- Cryptographic Failures (Fallas Criptográficas)
  - Token JWT en sessionStorage** ⚠️ **MEDIO**

**Problema:** `sessionStorage` es accesible por JavaScript → vulnerable a XSS si se inyecta un script malicioso.

**Riesgo:** Si un atacante logra inyectar `<script>` malicioso (ver A03), puede robar el token:

**Solución:**

  - Opción 1 (recomendado):** Usar cookies `HttpOnly` + `Secure` + `SameSite=Strict` → No accesibles desde JavaScript
  - Opción 2: Implementar Content Security Policy (CSP) estricto para bloquear scripts inline

**Estado:** ⚠️ VULNERABLE A XSS (aunque no se encontraron puntos de inyección directos)

  - Algoritmo JWT HS256 (simétrico) ⚠️ BAJO

**Configuración backend:**

# app/domain/auth/security.py

SECRET_KEY = "your-secret-key-here"  # ❌ Clave simétrica hardcoded

ALGORITHM = "HS256"

**Problema:** HS256 usa la misma clave para firmar y verificar tokens. Si la clave `SECRET_KEY` se filtra, un atacante puede forjar tokens JWT.

**Solución**: Usar RS256 (asimétrico) con par clave pública/privada.

**Estado**: ⚠️ RIESGO BAJO si la clave se mantiene segura (no commiteada en Git).

- Injection (Inyección)
  - XSS (Cross-Site Scripting) ✅ NO VULNERABLE

Pruebas realizadas:

| # | Entrada maliciosa | Componente | Resultado |

| 1 | `<script>alert('XSS')</script>` en campo `first_name` | Formulario Admin | React escapa automáticamente → Texto renderizado como string | ✅ Protegido |

| 2 | `<img src=x onerror=alert('XSS')>` en campo `description` | Formulario Aplicación | Escapado por React | ✅ Protegido |

| 3 | `"><script>alert(1)</script>` en campo email | Login | Validación Yup rechaza formato email inválido | ✅ Protegido |

| 4 | `javascript:alert('XSS')` en campo URL | Aplicación | React escapa, no ejecuta | ✅ Protegido |

**Análisis de código:**

# Búsqueda de patrones peligrosos

grep -r "dangerouslySetInnerHTML" frontend/src/

# ✅ Resultado: No matches found

grep -r "innerHTML" frontend/src/

# ✅ Resultado: No matches found

grep -r "eval(" frontend/src/

# ✅ Resultado: No matches found

**Conclusión:** ✅ El frontend NO es vulnerable a XSS porque:

1. React escapa automáticamente todas las variables en JSX
2. No se usa `dangerouslySetInnerHTML`

3. No se usa `eval()` ni `Function()`
4. No se usa `document.write()`

- SQL Injection** ✅ NO VULNERABLE

**Razón:** El backend usa SQLModel + SQLAlchemy con queries parametrizadas. No hay concatenación de strings SQL.

- Insecure Design (Diseño Inseguro)
  - Falta de autenticación multifactor (MFA) ⚠️ MEDIO

**Problema:** El sistema solo usa email + contraseña. No hay segundo factor (TOTP, SMS, hardware key).

**Riesgo:** Si se compromete la contraseña (phishing, keylogger, fuerza bruta), el atacante tiene acceso total.

**Solución:** Implementar TOTP (Google Authenticator) o WebAuthn.

**Estado:** ⚠️ FALTA IMPLEMENTAR

  - Sin rate limiting en login ❌ CRÍTICO

**Problema:** El endpoint `/api/v1/auth/login` no tiene rate limiting → vulnerable a ataques de fuerza bruta.

**Prueba:**

```
# Automatizar intentos de login
for i in {1..1000}; do
  curl -X POST http://localhost:8000/api/v1/auth/login \
    -H "Content-Type: application/json" \
    -d '{"email":"admin@test.com", "password":"pass'$i'"}'
done
# ❌ Ningún bloqueo después de 1000 intentos fallidos
```

**Solución:** Implementar Fail2Ban o usar middleware de rate limiting (e.g., `slowapi`).

**Estado:** ❌ VULNERABLE

- Sin validación de fuerza de contraseña ⚠️ MEDIO

**Problema:** El sistema solo valida longitud mínima (8 caracteres), no complejidad.

Contraseñas aceptadas:

- `12345678` ❌

- `aaaaaaaa` ❌

- `password` ❌

**Solución:** Validar complejidad (mayúsculas, minúsculas, números, símbolos) y comparar contra diccionario de contraseñas comunes.

**Estado:** ⚠️ VALIDACIÓN DÉBIL

- Security Misconfiguration (Configuración de Seguridad Incorrecta)
  - CORS permisivo ⚠️ MEDIO

**Configuración backend:**

```
# app/main.py
app.add_middleware(
  CORSMiddleware,
  allow_origins=["*"],  # ❌ Permite cualquier origen
  allow_credentials=True,
  allow_methods=["*"],
  allow_headers=["*"],
)
```

**Riesgo:** Permite peticiones desde cualquier dominio → vulnerable a CSRF.

**Solución:**

```
allow_origins=[
  "http://localhost:3000",
  "https://tu-dominio-produccion.com"
],
```

**Estado:** ⚠️ CONFIGURACIÓN PERMISIVA

- Debug mode en producción** ⚠️ **MEDIO**

**Verificar:**

```
# app/config.py
DEBUG = True  # ❌ Expone stack traces detallados
```

**Riesgo:** Los errores 500 revelan información sensible (rutas de archivos, consultas SQL, variables).

**Solución:**

```
DEBUG = os.getenv("DEBUG", "false").lower() == "true"
```

**Estado:** ⚠️ REVISAR CONFIGURACIÓN DE PRODUCCIÓN

- Sin Content Security Policy (CSP) ⚠️ MEDIO

**Problema:** El frontend no define CSP headers → no protege contra XSS inline.

Solución (nginx):

```
# frontend/nginx.conf
add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; font-src 'self'; connect-src 'self' http://localhost:8000;";
```

**Estado:** ⚠️ FALTA IMPLEMENTAR

- Vulnerable and Outdated Components (Componentes Vulnerables)

**Verificación de versiones:**

```
// frontend/package.json
{
  "dependencies": {
    "react": "^18.3.1",        // ✅ Última versión estable
    "@mui/material": "^6.7.0",   // ✅ Actualizado
    "@tanstack/react-query": "^5.30.0", // ✅ Actualizado
    "formik": "^2.4.6",        // ✅ Actualizado
    "yup": "^1.4.0"          // ✅ Actualizado
  }
}
```

**Estado:** ✅ DEPENDENCIAS ACTUALIZADAS

- Identification and Authentication Failures (Fallas de Autenticación)
    - Sin expiración forzada de sesión después de logout ⚠️ MEDIO

**Problema:** El logout solo elimina el token del `sessionStorage` en el cliente. El token JWT sigue siendo válido en el servidor hasta su expiración natural (60 min).

**Ataque:**

```
# 1. Usuario hace login
TOKEN=$(curl -s -X POST http://localhost:8000/api/v1/auth/login \
  -d '{"email":"admin@test.com","password":"pass"}' | jq -r '.access_token')


# 2. Usuario hace logout (frontend)
# sessionStorage se limpia
```

# 3. Atacante roba el token antes del logout (e.g., de logs del navegador)

# 4. Atacante usa el token después del logout

curl -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/administrators/

✅ FUNCIONA — El token sigue válido

**Solución:** Implementar blacklist de tokens o usar refresh tokens con revocación.

**Estado:** ⚠️ VULNERABLE

Sin bloqueo de cuenta después de intentos fallidos ❌ CRÍTICO

**Problema:** El sistema no bloquea cuentas después de X intentos fallidos → permite fuerza bruta ilimitada.

**Solución:**

- Después de 5 intentos fallidos: Bloquear cuenta por 15 minutos

- Después de 10 intentos: Bloquear permanentemente (requiere admin para desbloquear)

**Estado:** ❌ FALTA IMPLEMENTAR

    - Sin verificación de email ⚠️ MEDIO

**Problema:** Al crear un usuario, no se verifica que el email sea real → permite registros masivos con emails falsos.

**Solución:** Enviar email de confirmación con token de activación.

**Estado:** ⚠️ FALTA IMPLEMENTAR

- Software and Data Integrity Failures (Fallas de Integridad)
    - Sin Subresource Integrity (SRI) para CDNs ⚠️ BAJO

**Verificar:**

<!-- frontend/index.html -->

<link

 rel="stylesheet"

```
href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@300..700&display=swap"
/>
<!-- ❌ Sin atributo integrity -->
```

**Riesgo:** Si el CDN es comprometido, puede servir CSS malicioso.

**Solución:**

```
<link
  rel="stylesheet"
  href="https://fonts.googleapis.com/..."
  integrity="sha384-..."
  crossorigin="anonymous"
/>
```

**Estado:** ⚠️ FALTA SRI

- Sin firma digital de actualizaciones ⚠️ BAJO

**Problema:** No hay mecanismo para verificar la integridad de las actualizaciones del frontend.

**Solución:** Implementar firma digital de builds con GPG o verificación de hash SHA256.

**Estado:** ⚠️ FALTA IMPLEMENTAR

- Security Logging and Monitoring Failures (Fallas de Registro y Monitoreo)
  - Sin logs de auditoría en backend** ❌ **CRÍTICO**

**Problema:** El backend no registra eventos de seguridad (login fallido, cambios administrativos, eliminaciones).

Logs deseados:

2026-04-20 14:32:10 | WARN | LOGIN_FAILED | user=admin@test.com | ip=192.168.1.100

2026-04-20 14:35:22 | INFO | ADMIN_DELETED | actor=admin1 | target=admin2 | reason="Duplicado"

2026-04-20 14:40:15 | WARN | PASSWORD_CHANGE_FAILED | user=user@test.com | reason="Contraseña actual incorrecta"

**Solución:** Implementar logging estructurado con `structlog` y enviar a SIEM (Elasticsearch, Splunk).

**Estado:** ❌ FALTA IMPLEMENTAR

- Sin monitoreo de accesos anómalos ⚠️ MEDIO

Ejemplos de anomalías no detectadas:

- 100 intentos de login en 1 minuto desde la misma IP

- Acceso desde país diferente al habitual

- Cambio de contraseña seguido de eliminación masiva de datos

**Solución:** Implementar alertas automatizadas (e.g., con Prometheus + Grafana).

**Estado:** ⚠️ FALTA IMPLEMENTAR

- Server-Side Request Forgery (SSRF)

Sin endpoints que realicen peticiones a URLs externas** ✅ NO VULNERABLE

**Análisis:** El backend no tiene funcionalidades que acepten URLs como parámetros y hagan peticiones HTTP.

**Endpoints revisados:**

- `POST /api/v1/applications/` → Campo `url` solo se almacena, no se consulta

- `POST /api/v1/devices/` → Sin campos de URL

Estado: ✅ NO VULNERABLE

**Errores y Problemas Encontrados**

- Errores funcionales

| # | Problema | Situación | Código error | Severidad | Estado |

| 1 | Backend sin autorización en `/managers/` | Usuario normal crea gerente con cURL | 201 (debería ser 403) | ❌ CRÍTICA | Sin fix |

| 2 | Backend sin autorización en `/devices/` | Usuario normal elimina dispositivo | 204 (debería ser 403) | ❌ CRÍTICA | Sin fix |

| 3 | Admin puede auto-eliminarse con API directa | `DELETE /administrators/{self_id}` | 204 | ⚠️ ALTA | Protección solo UI |

| 4 | Token JWT válido después de logout | Usar token robado post-logout | 200 (debería invalidarse) | ⚠️ ALTA | Sin fix |

| 5 | Sin rate limiting en login | 1000 intentos de fuerza bruta | N/A | ❌ CRÍTICA | Sin fix |

| 6 | CORS permite cualquier origen | Petición desde `evil.com` | N/A | ⚠️ MEDIA | Config permisiva |

| 7 | UX pobre en tickets: `user_role_id` texto libre | Usuario debe conocer UUID | N/A | ⚠️ MEDIA | Falta select |

| 8 | UX pobre en tickets: `manager_service_id` texto libre | Usuario debe conocer UUID | N/A | ⚠️ MEDIA | Falta select |

- Errores de UI

| # | Problema | Situación | Efecto visual | Severidad | Estado |

| 1 | Diálogo no se cierra si error de red | Submit fallido → Diálogo abierto | Usuario bloqueado | ⚠️ MEDIA | Manejado con Alert |

| 2 | Tabla sin mensaje "Sin datos" | Lista vacía | Tabla en blanco | ⚠️ BAJA | MUI muestra "No rows" |

| 3 | Spinner de carga muy rápido | Petición responde en <100ms | Spinner flashea | ⚠️ BAJA | Normal |

**Errores de validación**

| # | Campo | Validación faltante | Impacto | Estado |

| 1 | `password` | Sin validación de complejidad | Acepta "12345678" | ⚠️ MEDIA | Solo longitud |

| 2 | `curp` | Regex permite formatos inválidos | Backend no valida dígito verificador | ⚠️ BAJA | Regex básico |

| 3 | `rfc` | Sin validación de formato | Acepta cualquier string | ⚠️ MEDIA | Sin regex |

| 4 | `postal_code` | Sin validación de rango (México: 01000-99999) | Acepta "00000" | ⚠️ BAJA | Sin validación |

| 5 | `birth_date` | Permite fechas futuras | Usuario puede nacer en 2050 | ⚠️ BAJA | Sin `max={today}` |

**Funcionalidades Faltantes**

- Desde el reporte de estado

| # | Funcionalidad | Descripción | Bloqueador | Prioridad |

| 1 | Asignar usuarios a servicios | Diálogo UI completo pero sin endpoints backend | Falta `GET /services/{id}/users`, `POST /services/{id}/users/{user_id}`, `DELETE /services/{id}/users/{user_id}` | 🔴 ALTA |

| 2 | Select de User Roles en tickets | Campo texto libre — UX pobre | Falta `GET /user-roles/` | 🟡 MEDIA |

| 3 | Select de Manager Services en tickets | Campo texto libre — UX pobre | Falta `GET /manager-services/` | 🟡 MEDIA |

**Mejoras de seguridad**

| # | Funcionalidad | Descripción | Impacto | Prioridad |

| 1 | Rate limiting en login | Bloquear después de 5 intentos fallidos | Previene fuerza bruta | 🔴 CRÍTICA |

| 2 | Autenticación multifactor (MFA) | TOTP o SMS | Protección contra phishing | 🟡 MEDIA |

| 3 | Blacklist de tokens JWT | Invalidar tokens al hacer logout | Previene uso de tokens robados | 🟡 MEDIA |

| 4 | Logs de auditoría | Registrar acciones administrativas | Forense y cumplimiento | 🟡 MEDIA |

| 5 | Validación de fuerza de contraseña | Requerir mayúsculas + números + símbolos | Mejora seguridad de cuentas | 🟢 BAJA |

| 6 | Verificación de email | Token de activación por email | Previene registros falsos | 🟢 BAJA |

**Mejoras de UX**

| # | Funcionalidad | Descripción | Impacto | Prioridad |

| 1 | Búsqueda en tablas | Filtrar por nombre/email/CURP | Mejor navegación | 🟡 MEDIA |

| 2 | Exportar datos a CSV/Excel | Botón "Exportar" en tablas | Reportes para usuarios | 🟡 MEDIA |

| 3 | Notificaciones push | Alertas en tiempo real (nuevo ticket, cambio de estado) | Mejor comunicación | 🟢 BAJA |

| 4 | Modo oscuro (Dark mode) | Toggle en AppBar | Confort visual | 🟢 BAJA |

| 5 | Historial de cambios | Ver quién editó qué y cuándo | Trazabilidad | 🟡 MEDIA |

| 6 | Paginación mejorada | Seleccionar cantidad de filas (10/25/50/100) | Flexibilidad | 🟢 BAJA |

**Conclusiones y Recomendaciones**

- Resumen de estado del sistema

| Aspecto | Estado | Calificación |

| **Funcionalidad CRUD** | Todas las entidades principales implementadas | ✅ 9/10 |

| **Autenticación JWT** | Funcional con roles y permisos en UI | ✅ 8/10 |

| **Autorización backend** | ⚠️ Vulnerable en varios endpoints | ❌ 4/10 |

| **Validación de formularios** | Completa con Formik + Yup | ✅ 9/10 |

| **Responsividad** | Adaptativo a móvil/tablet/desktop | ✅ 9/10 |

| **Accesibilidad** | Buena semántica, falta ARIA labels | ⚠️ 7/10 |

| **Rendimiento** | Bundle aceptable, sin lazy loading | ⚠️ 7/10 |

| **Seguridad (OWASP)** | Múltiples vulnerabilidades críticas | ❌ 3/10 |

| **UX** | Intuitivo, falta búsqueda y exportación | ⚠️ 8/10 |

| **Documentación** | Reporte técnico completo | ✅ 10/10 |

**Calificación general:** ⚠️ **7.2/10** — Sistema funcional pero con riesgos de seguridad críticos.

## Prioridades de corrección

🔴 CRÍTICO (Arreglar antes de producción)

1. Autorización backend: Agregar decoradores OSO en `/managers/`, `/devices/`, `/services/`
2. Rate limiting: Implementar límite de 5 intentos de login por IP/minuto
3. Validación de auto-eliminación: Backend debe prevenir que un admin se elimine a sí mismo
4. CORS restrictivo: Cambiar `allow_origins=["*"]` a lista blanca específica

🟡 IMPORTANTE (Arreglar en próximo sprint)

5. Blacklist de tokens JWT: Invalidar tokens al hacer logout
6. Logs de auditoría: Registrar acciones administrativas (login, CRUD, cambios de permisos)

7. Validación de fuerza de contraseña: Requerir complejidad (mayúsculas + números + símbolos)
8. Bloqueo de cuenta: Bloquear después de 5 intentos fallidos de login

🟢 MEJORAS (Plan a mediano plazo)

9. Autenticación multifactor (MFA): TOTP o SMS para admin master
10. Funcionalidad de asignación usuarios-servicios: Desarrollar endpoints backend faltantes
11. Búsqueda y exportación en tablas: Filtros y exportar a CSV/Excel
12. Code splitting: Lazy loading de rutas para reducir bundle inicial

**Riesgos principales**

❌ Seguridad backend:

- Endpoints críticos sin autorización permiten escalada de privilegios

- Sin rate limiting → vulnerable a fuerza bruta

- Sin logs de auditoría → imposible detectar ataques

❌ Gestión de sesiones:

- Tokens válidos post-logout

- Sin MFA para cuentas privilegiadas

- Token en `sessionStorage` vulnerable a XSS

❌ Validación de contraseñas:

- Acepta contraseñas débiles (`12345678`)

- Sin comparación con diccionario de contraseñas comunes